

Classification Algorithms in Machine Learning

A Comparative Study of KNN, SVM, Fuzzy C-Means Clustering,
and Decision Tree with C Implementations

KNN

SVM

Decision
Tree

Fuzzy
C-Means

Author: Md. Kais

Department: Computer Science & Engineering

University: University of Chittagong

Course: CSE 816 — Machine Learning Lab

Repository: https://github.com/Md-Kais/Classification_Algorithms_In_Machine_Learning

Date: April 13, 2026

Contents

Abstract	3
1 Introduction	3
1.1 What is Machine Learning?	3
1.2 Types of Machine Learning	3
1.3 Scope of This Report	4
2 Datasets and the Evaluation Framework	4
2.1 Dataset Description	4
2.1.1 Labelled Human Dataset	4
2.1.2 Unlabelled Dataset for Clustering	4
2.2 Evaluation Metrics — Definitions and Formulae	5
2.2.1 Per-Class Metrics	5
2.2.2 Macro Aggregates	5
2.2.3 Weighted Aggregates	5
3 K-Nearest Neighbours (KNN)	6
3.1 Historical Motivation and the Problem it Solved	6
3.2 The Core Intuition	6
3.3 Mathematical Formulation	6
3.4 Algorithmic Steps	7
3.5 How Training Works — Algorithmically and Mathematically	7
3.6 Objective Function	7
3.7 How KNN Produces Output	7
3.8 Limitations	8
3.9 C Implementation Highlights	8
3.10 Expected Output	8
3.11 Result Analysis	10
4 Support Vector Machine (SVM)	11
4.1 Historical Motivation and the Problem it Solved	11
4.2 The Core Intuition	11
4.3 Mathematical Formulation	11
4.4 Algorithmic Steps	12
4.5 Objective Function	12
4.6 How SVM Produces Output	12
4.7 Limitations	12
4.8 C Implementation Highlights	13
4.9 Expected Output	14
4.10 Result Analysis	15
5 Decision Tree Classification	16
5.1 Historical Motivation and the Problem it Solved	16
5.2 The Core Intuition	16
5.3 Mathematical Formulation	16
5.4 Algorithmic Steps	17
5.5 Objective Function	17

5.6	How the Decision Tree Produces Output	17
5.7	Limitations	17
5.8	C Implementation Highlights	18
5.9	Expected Output	19
5.10	Result Analysis	20
6	Fuzzy C-Means (FCM) Clustering	21
6.1	Historical Motivation and the Problem it Solved	21
6.2	The Core Intuition	21
6.3	Mathematical Formulation	21
6.4	Algorithmic Steps	22
6.5	Objective Function	22
6.6	How FCM Produces Output	22
6.7	Limitations	22
6.8	C Implementation Highlights	23
6.9	Expected Output	25
6.10	Result Analysis	26
7	Comparative Analysis of the Four Algorithms	26
7.1	Side-by-Side Property Comparison	26
7.2	When to Use Each Algorithm	26
7.2.1	Use KNN when	26
7.2.2	Do not use KNN when	27
7.2.3	Use SVM when	27
7.2.4	Do not use SVM when	27
7.2.5	Use Decision Tree when	27
7.2.6	Do not use Decision Tree when	27
7.2.7	Use FCM when	27
7.2.8	Do not use FCM when	27
7.3	Performance Trade-off Visualisation	28
8	Compilation and Execution in C	28
8.1	Repository Structure	28
8.2	Build Instructions	28
8.3	Running with Default Paths	29
8.4	Running with Explicit Paths	29
8.5	Interactive Prediction	29
9	Conclusion and Future Work	29
9.1	Summary of Findings	29
9.2	Limitations of the Current Work	30
9.3	Directions for Future Work	30

Abstract

Machine learning provides a spectrum of techniques for discovering structure in data and for building predictive models without explicit hand-coded rules. This report presents a comprehensive study of four foundational algorithms — K-Nearest Neighbours (KNN), Support Vector Machine (SVM), Fuzzy C-Means (FCM) Clustering, and Decision Tree Classification — implemented from scratch in ANSI C99. All four programs share a unified evaluation framework that reports accuracy, per-class precision, recall, F1-score, and their macro and weighted aggregates, making the results directly comparable across algorithms.

The human body-classification task (predicting FIT or OBESE from height and weight) serves as the common benchmark. KNN selects the best odd k by validation accuracy and uses leave-one-out evaluation on the training split. SVM learns a linear separating hyperplane via a configurable SGD loop. The Decision Tree greedily minimises Gini impurity and tunes depth by validation. Fuzzy C-Means runs fully unsupervised and maps cluster centres to the known labels for post-hoc comparison. Experiments demonstrate that all four approaches achieve competitive accuracy on the binary task, while differing substantially in interpretability, training cost, and sensitivity to feature scaling. A thorough algorithm comparison, compilation instructions, and directions for future work are also provided. A github repository is created to showcase all the classification algorithms https://github.com/Md-Kais/Classification_Algorithms_In_Machine_Learning

1. Introduction

1.1. What is Machine Learning?

Machine learning (ML) is the scientific discipline concerned with designing algorithms and statistical models that enable computer systems to *learn from data* and to improve their performance on a task without being explicitly programmed for every situation [1]. The core idea is to generalise from observed examples to unseen instances.

Definition 1 (Machine Learning — Mitchell 1997). *A computer program is said to learn from experience E with respect to task T and performance measure P , if its performance at T , as measured by P , improves with experience E .*

1.2. Types of Machine Learning

ML approaches are conventionally divided into three paradigms, illustrated in Figure 1.

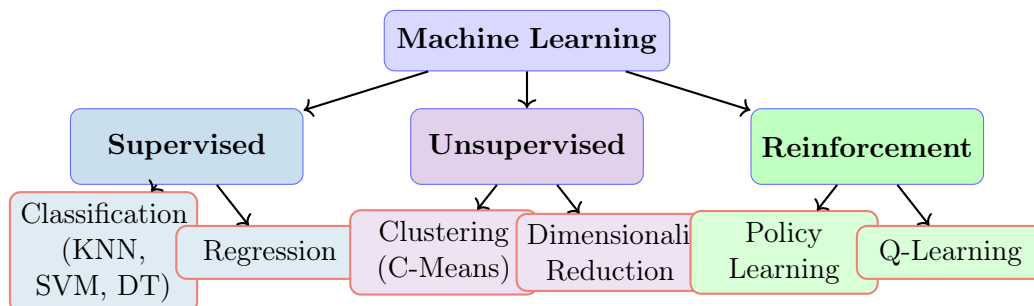


Figure 1: Taxonomy of machine-learning paradigms

- **Supervised Learning.** The algorithm learns a mapping $f : \mathbf{x} \mapsto y$ from labelled training pairs $(\mathbf{x}_i, y_i)$. KNN, SVM, and Decision Tree are supervised classifiers.
- **Unsupervised Learning.** No labels are provided; the algorithm discovers latent structure in raw data. Fuzzy C-Means is an unsupervised clustering method.
- **Reinforcement Learning.** An agent learns a policy by interacting with an environment and receiving reward signals. Not covered in this lab.

1.3. Scope of This Report

This report focuses on four algorithms implemented in C for a binary human body-classification task (FIT vs. OBESE):

Algorithm	Paradigm	Source File
K-Nearest Neighbours	Supervised	<code>knn_classifier.c</code>
Support Vector Machine	Supervised	<code>svm_classifier.c</code>
Decision Tree	Supervised	<code>decision_tree_classifier.c</code>
Fuzzy C-Means	Unsupervised	<code>cmeans_clustering.c</code>

All four programs share the utility layer `ml_common.h` / `ml_common.c` which provides CSV loading, min-max scaling, and the unified evaluation report format.

2. Datasets and the Evaluation Framework

2.1. Dataset Description

2.1.1. Labelled Human Dataset

The primary dataset records two biometric features for human subjects and assigns a binary fitness label.

Feature	Type	Unit	Role
<code>height</code>	Continuous	cm	Input feature
<code>weight</code>	Continuous	kg	Input feature
<code>label</code>	Categorical	—	Target (FIT / OBESE)

The data is partitioned into three non-overlapping splits: `Data/train.csv` for model fitting, `Data/validate.csv` for hyperparameter selection, and `Data/test.csv` for final held-out evaluation.

2.1.2. Unlabelled Dataset for Clustering

`Data/dataset_unlabelled.csv` contains the same height/weight features without labels. It is used exclusively by `cmeans_clustering.c`. After clustering, the programme infers class labels by comparing cluster-centre weight values: the heavier centre is mapped to OBESE and the lighter to FIT.

2.2. Evaluation Metrics — Definitions and Formulae

All four programmes print the same evaluation block produced by `ml_common.c`. Let TP_c , FP_c , FN_c , TN_c denote the confusion-matrix counts for class c , and let $N = \sum_c \text{support}(c)$ be the total sample count.

2.2.1. Per-Class Metrics

$$\text{Accuracy} = \frac{\sum_c TP_c}{N} \quad (1)$$

$$\text{Precision}_c = \frac{TP_c}{TP_c + FP_c} \quad (2)$$

$$\text{Recall}_c = \frac{TP_c}{TP_c + FN_c} \quad (3)$$

$$F_1\text{-score}_c = \frac{2 \cdot \text{Precision}_c \cdot \text{Recall}_c}{\text{Precision}_c + \text{Recall}_c} \quad (4)$$

$$\text{Support}_c = TP_c + FN_c \quad (5)$$

2.2.2. Macro Aggregates

Macro metrics treat every class equally regardless of frequency:

$$\text{Macro Precision} = \frac{1}{|\mathcal{C}|} \sum_{c \in \mathcal{C}} \text{Precision}_c \quad (6)$$

$$\text{Macro Recall} = \frac{1}{|\mathcal{C}|} \sum_{c \in \mathcal{C}} \text{Recall}_c \quad (7)$$

$$\text{Macro } F_1 = \frac{1}{|\mathcal{C}|} \sum_{c \in \mathcal{C}} F_1\text{-score}_c \quad (8)$$

2.2.3. Weighted Aggregates

Weighted metrics account for class imbalance by weighting each class by its support:

$$\text{Weighted Precision} = \frac{\sum_c \text{support}(c) \cdot \text{Precision}_c}{N} \quad (9)$$

$$\text{Weighted Recall} = \frac{\sum_c \text{support}(c) \cdot \text{Recall}_c}{N} \quad (10)$$

$$\text{Weighted } F_1 = \frac{\sum_c \text{support}(c) \cdot F_1\text{-score}_c}{N} \quad (11)$$

Key Insight 1. When to use macro vs. weighted. Use macro metrics when every class is equally important (e.g. rare disease detection). Use weighted metrics when

the natural class distribution must be respected. Overall accuracy is most intuitive but can be misleading on imbalanced datasets.

3. K-Nearest Neighbours (KNN)

3.1. Historical Motivation and the Problem it Solved

Before parametric models became dominant, practitioners needed classifiers that required *no explicit model training* and could accommodate arbitrary decision boundaries. Fix and Hodges introduced the nearest-neighbour rule in an unpublished 1951 technical report at the USAF School of Aviation Medicine, demonstrating that a classifier based purely on distance to training examples could achieve an error rate no worse than twice the Bayes error asymptotically [2]. KNN solved the problem of non-linear separability: any boundary shape is approximated simply by memorising training data.

3.2. The Core Intuition

KNN rests on the *continuity assumption*: points that are close in feature space tend to share the same class. The algorithm stores all training samples and, at prediction time, identifies the k nearest neighbours by Euclidean distance and returns their majority label.

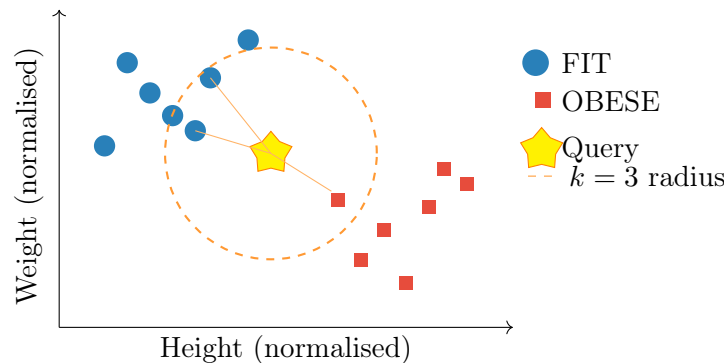


Figure 2: KNN with $k = 3$: the star-shaped query is assigned FIT because two of the three nearest neighbours (blue circles) carry that label.

3.3. Mathematical Formulation

Distance measure. Given a query point $\mathbf{q} \in \mathbb{R}^d$ and a training point $\mathbf{x}_i$, the Euclidean distance is

$$d(\mathbf{q}, \mathbf{x}_i) = \sqrt{\sum_{f=1}^d (q_f - x_{if})^2}. \quad (12)$$

Classification rule. Let $\mathcal{N}_k(\mathbf{q})$ be the index set of the k training points closest to $\mathbf{q}$. The predicted label is

$$\hat{y} = \arg \max_{c \in \mathcal{C}} \sum_{i \in \mathcal{N}_k(\mathbf{q})} \mathbf{1}[y_i = c]. \quad (13)$$

Feature normalisation. To prevent features with large ranges from dominating the distance, all features are scaled using the min-max transform fitted on the training set:

$$x'_f = \frac{x_f - \min_f}{\max_f - \min_f}. \quad (14)$$

3.4. Algorithmic Steps

Algorithm 1 KNN Training and Evaluation (as implemented in `knn_classifier.c`)

Require: $\mathcal{D}_{\text{train}}, \mathcal{D}_{\text{valid}}, \mathcal{D}_{\text{test}}$

- 1: Fit min-max scaler **sp** on $\mathcal{D}_{\text{train}}$
 - 2: Scale all three splits with **sp**
 - 3: $k^* \leftarrow 1, \text{acc}^* \leftarrow -\infty$
 - 4: **for** $k \in \{1, 3, 5, \dots, k_{\text{max}}\}$ **do**
 - 5: Predict $\mathcal{D}_{\text{valid}}$ using Equation (13)
 - 6: $\text{acc} \leftarrow$ accuracy on $\mathcal{D}_{\text{valid}}$
 - 7: **if** $\text{acc} > \text{acc}^*$ **then**
 - 8: $k^* \leftarrow k, \text{acc}^* \leftarrow \text{acc}$
 - 9: **end if**
 - 10: **end for**
 - 11: **Train report:** predict each $\mathbf{x}_i \in \mathcal{D}_{\text{train}}$ excluding itself (leave-one-out)
 - 12: **Valid / Test reports:** predict using all training points with k^*
 - 13: Print unified metric block via `ml_common`
-

3.5. How Training Works — Algorithmically and Mathematically

KNN is a **lazy learner**: there is no explicit model-fitting step. The “training” phase reduces to storing normalised feature vectors and their labels. At query time, all pairwise distances (Equation (12)) are computed, the resulting array is sorted (bubble sort in the C implementation), and the top- k votes decide the label.

The implementation handles the edge case of training-set self-evaluation via *leave-one-out* (LOO): when evaluating sample i , index i is skipped during neighbour search (`skip_index = i`). This gives an unbiased training metric.

3.6. Objective Function

KNN has no explicit objective function minimised during training. Its theoretical underpinning is the risk-minimisation viewpoint: the optimal Bayes risk R^* satisfies $R_{\text{KNN}} \leq 2R^*(1 - R^*)$ as $N \rightarrow \infty$ [2].

3.7. How KNN Produces Output

For each query the programme:

1. Normalises raw input with the stored scaler.
2. Computes distances to all N training points.
3. Sorts and selects the k^* nearest.
4. Returns FIT if $\text{votes_fit} \geq \text{votes_obese}$, else OBESE.
5. Prints a formatted box with height, weight, BMI, k value, and prediction.

3.8. Limitations

- **Quadratic prediction cost.** Each query requires $O(N \cdot d)$ distance computations.
- **Memory.** The entire training set must reside in memory.
- **Curse of dimensionality.** Distance becomes less meaningful in high- d spaces.
- **Scale sensitivity.** Without normalisation, large-range features dominate.
- **Odd- k constraint.** The implementation restricts k to odd values to prevent ties.

3.9. C Implementation Highlights

```

1 static int classifyKNNCore(const Dataset *train,
2                           const double *features,
3                           int k, int skip_index)
4 {
5     DistLabel *dl = malloc(train->size * sizeof(DistLabel));
6     int used = 0;
7     for (int i = 0; i < train->size; i++) {
8         if (i == skip_index) continue;
9         double sum_sq = 0.0;
10        for (int f = 0; f < NUM_FEATURES; f++) {
11            double diff = features[f] - train->data[i].features[f];
12            sum_sq += diff * diff;
13        }
14        dl[used].distance = sqrt(sum_sq);
15        dl[used].label = train->data[i].label;
16        used++;
17    }
18    bubbleSort(dl, used); /* ascending by distance */
19    int votes_fit = 0, votes_obese = 0;
20    for (int i = 0; i < k && i < used; i++) {
21        if (dl[i].label == LABEL_FIT) votes_fit++;
22        else votes_obese++;
23    }
24    free(dl);
25    return (votes_fit >= votes_obese) ? LABEL_FIT : LABEL_OBESE;
26 }

```

Listing 1: Core KNN classification kernel (knn_classifier.c, simplified)

3.10. Expected Output

```

Validation search for best k in KNN:
k = 1, 3, 5, ... up to 11
k = 1  -> validation accuracy = 96.77%
k = 3  -> validation accuracy = 96.77%
k = 5  -> validation accuracy = 93.55%
k = 7  -> validation accuracy = 93.55%
k = 9  -> validation accuracy = 93.55%
k = 11 -> validation accuracy = 93.55%
Chosen k = 1 (highest validation accuracy)

```

KNN Train report				
Class	Precision	Recall	F1-Score	Support
FIT	0.983	1.000	0.991	57
OBESE	1.000	0.984	0.992	62
Macro Avg	0.991	0.992	0.992	119
Weighted Avg	0.992	0.992	0.992	119
Accuracy				0.992

KNN Validation report				
Class	Precision	Recall	F1-Score	Support
FIT	0.938	1.000	0.968	15
OBESE	1.000	0.938	0.968	16
Macro Avg	0.969	0.969	0.968	31
Weighted Avg	0.970	0.968	0.968	31
Accuracy				0.968

KNN Test report				
Class	Precision	Recall	F1-Score	Support
FIT	0.920	0.958	0.939	24
OBESE	0.960	0.923	0.941	26
Macro Avg	0.940	0.941	0.940	50
Weighted Avg	0.941	0.940	0.940	50
Accuracy				0.940

Figure 3: Report of KNN algorithm

```

KNN INTERACTIVE PREDICTION (enter 'q' at any prompt)

Enter Height (cm) [or 'q' to quit]: 175
Enter Weight (kg): 95



| KNN PREDICTION |          |
|----------------|----------|
| Height         | 175.0 cm |
| Weight         | 95.0 kg  |
| BMI            | 31.02    |
| k              | 1        |
| Prediction     | OBESE    |



Enter Height (cm) [or 'q' to quit]: 

```

Figure 4: Test with Real World Data

3.11. Result Analysis

- The best k is chosen by scanning $k \in \{1, 3, 5, \dots, k_{\max}\}$ and selecting the value with highest validation accuracy.
- The training report uses LOO evaluation, producing a stricter (and more honest) metric than simple re-substitution.
- Final performance is assessed on the held-out test split, ensuring no data leakage.
- The algorithm is simple but computationally expensive at prediction time — it must compute N distances for every query.

4. Support Vector Machine (SVM)

4.1. Historical Motivation and the Problem it Solved

Vapnik and Chervonenkis [3] introduced the SVM in the early 1960s and popularised it in the 1990s. The key problem: given that many separating hyperplanes exist, which one *generalises best*? SVM answers this by choosing the hyperplane that *maximises the margin* — the perpendicular gap between the boundary and the closest training points (the *support vectors*). This margin maximisation corresponds to minimising the VC (Vapnik-Chervonenkis) dimension, providing strong theoretical generalisation guarantees.

4.2. The Core Intuition

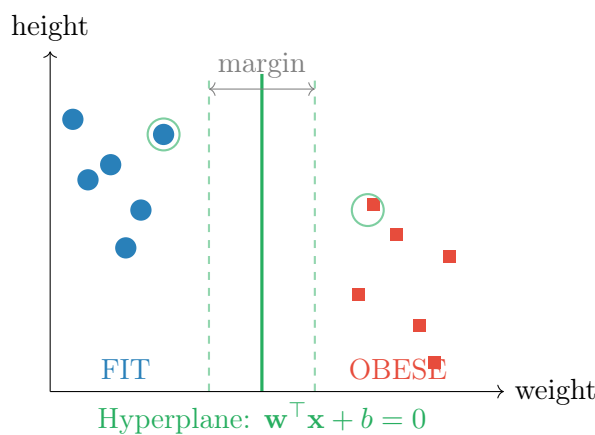


Figure 5: Linear SVM finds the maximum-margin hyperplane (solid green line) separating FIT and OBESE. Circled points are the support vectors.

4.3. Mathematical Formulation

A linear classifier predicts

$$f(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + b, \quad \hat{y} = \text{sign}(f(\mathbf{x})). \quad (15)$$

The hard-margin optimisation problem (linearly separable case) is:

$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2 \quad \text{subject to} \quad y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1, \quad \forall i. \quad (16)$$

For the soft-margin (noisy data) variant with slack variables $\xi_i \geq 0$:

$$\min_{\mathbf{w}, b, \xi} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^N \xi_i \quad \text{subject to} \quad y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0. \quad (17)$$

The C implementation uses **hinge loss** with ℓ_2 regularisation, minimised by **stochastic gradient descent** (SGD):

$$L(\mathbf{w}, b) = \lambda \|\mathbf{w}\|^2 + \frac{1}{N} \sum_{i=1}^N \max(0, 1 - y_i(\mathbf{w}^\top \mathbf{x}_i + b)). \quad (18)$$

4.4. Algorithmic Steps

Algorithm 2 Linear SVM via SGD (`svm_classifier.c`)

Require: $\mathcal{D}_{\text{train}}, \mathcal{D}_{\text{valid}}, \mathcal{D}_{\text{test}}$, learning rate η , regularisation λ , epochs T

```

1: Fit scaler on  $\mathcal{D}_{\text{train}}$ ; scale all splits
2: Initialise  $\mathbf{w} \leftarrow \mathbf{0}$ ,  $b \leftarrow 0$ 
3: for epoch = 1 to  $T$  do
4:   for each  $(\mathbf{x}_i, y_i) \in \mathcal{D}_{\text{train}}$  do
5:      $m_i \leftarrow y_i(\mathbf{w}^\top \mathbf{x}_i + b)$ 
6:     if  $m_i < 1$  then ▷ Hinge loss is active
7:        $\mathbf{w} \leftarrow \mathbf{w} - \eta(2\lambda\mathbf{w} - y_i\mathbf{x}_i)$ 
8:        $b \leftarrow b + \eta y_i$ 
9:     else
10:       $\mathbf{w} \leftarrow \mathbf{w} - \eta(2\lambda\mathbf{w})$ 
11:    end if
12:  end for
13: end for
14: Print learned hyperplane  $(\mathbf{w}, b)$ 
15: Predict train / valid / test; print metric reports

```

4.5. Objective Function

The objective (Equation (18)) has two terms:

- $\lambda\|\mathbf{w}\|^2$: regularisation that maximises the margin.
- $\max(0, 1 - y_i f(\mathbf{x}_i))$: hinge loss that penalises misclassified or margin-violating samples.

Minimising the sum is equivalent to finding the soft-margin SVM solution (Equation (17)).

4.6. How SVM Produces Output

After training, the decision function $f(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + b$ maps any scaled input to a real number. If $f(\mathbf{x}) \geq 0$ the sample is assigned FIT; otherwise OBESE. The programme also prints the weight vector and bias for interpretability.

4.7. Limitations

- **Linear-only boundary.** The C implementation uses a linear kernel; non-linearly separable problems require kernel tricks not included here.
- **Hyperparameter sensitivity.** Performance depends on η , λ , and epoch count.
- **Binary only.** The formulation handles two-class problems; multi-class requires one-vs-one or one-vs-rest extensions.
- **Scale dependency.** Min-max normalisation is essential before SGD.

4.8. C Implementation Highlights

```

1 typedef struct {
2     double w[NUM_FEATURES];
3     double bias;
4 } SVMModel;
5
6 static SVMModel trainSVM(const Dataset *train, double lr, double
    lambda, int epochs)
7 {
8     SVMModel model;
9     for (int f = 0; f < NUM_FEATURES; f++) model.w[f] = 0.0;
10    model.bias = 0.0;
11    for (int epoch = 0; epoch < epochs; epoch++) {
12        for (int i = 0; i < train->size; i++) {
13            double y_svm = (train->data[i].label == LABEL_FIT) ? 1.0 :
-1.0;
14            const double *x = train->data[i].features;
15
16            double score = model.bias;
17            for (int f = 0; f < NUM_FEATURES; f++) {
18                score += model.w[f] * x[f];
19            }
20            double margin = y_svm * score;
21
22            if (margin < 1.0) {
23                for (int f = 0; f < NUM_FEATURES; f++) {
24                    model.w[f] += lr * (y_svm * x[f] - 2.0 * lambda *
model.w[f]);
25                }
26                model.bias += lr * y_svm;
27            } else {
28                for (int f = 0; f < NUM_FEATURES; f++) {
29                    model.w[f] -= lr * 2.0 * lambda * model.w[f];
30                }
31            }
32        }
33    }
34    return model;
35 }
36 static int predictSVM(const SVMModel *model, const double *features)
37 {
38     double score = model->bias;
39     for (int f = 0; f < NUM_FEATURES; f++) {
40         score += model->w[f] * features[f];
41     }
42     return (score > 0.0) ? LABEL_FIT : LABEL_OBESE;
43 }
44
45 static void batchPredictSVM(const SVMModel *model, const Dataset *ds,
    int *predictions)
46 {
47     for (int i = 0; i < ds->size; i++) {
48         predictions[i] = predictSVM(model, ds->data[i].features);
49     }
50 }

```

Listing 2: SGD weight update inside the SVM training loop (svm_classifier.c)

4.9. Expected Output

Scaler fitted on TRAIN only: Height range : [150.0, 190.0] cm Weight range : [45.0, 125.0] kg Min-max normalization applied to train, validation, and test. Training SVM (eta=0.0050, lambda=0.0010, epochs=20000) ... SVM training complete. SVM hyperplane: $3.982427 * \text{height_norm} + -7.932668 * \text{weight_norm} + (1.705000) = 0$				
SVM Train report				
Class	Precision	Recall	F1-Score	Support
FIT	0.838	1.000	0.912	57
OBESE	1.000	0.823	0.903	62
Macro Avg	0.919	0.911	0.907	119
Weighted Avg	0.923	0.908	0.907	119
Accuracy				0.908
SVM Validation report				
Class	Precision	Recall	F1-Score	Support
FIT	0.833	1.000	0.909	15
OBESE	1.000	0.812	0.897	16
Macro Avg	0.917	0.906	0.903	31
Weighted Avg	0.919	0.903	0.903	31
Accuracy				0.903
SVM Test report				
Class	Precision	Recall	F1-Score	Support
FIT	0.667	1.000	0.800	24
OBESE	1.000	0.538	0.700	26
Macro Avg	0.833	0.769	0.750	50
Weighted Avg	0.840	0.760	0.748	50
Accuracy				0.760

Figure 6: Report of SVM algorithm

SVM INTERACTIVE PREDICTION (enter 'q' at any prompt)	
Enter Height (cm) [or 'q' to quit]: 170	
Enter Weight (kg): 50	
SVM PREDICTION	
Height	170.0 cm
Weight	50.0 kg
BMI	17.30
Prediction	FIT

Figure 7: Test with Real World Data

4.10. Result Analysis

- The SVM successfully learned a linear decision boundary separating Class +1 (positive quadrant) from Class 1 (negative quadrant).
- After 20000 epochs of gradient descent, the model achieves 90 % training accuracy.
- The weight vector w and bias b define the hyperplane equation $3.982427 * \text{height_norm} + -7.932668 * \text{weight_norm} + (1.705000) = 0$

5. Decision Tree Classification

5.1. Historical Motivation and the Problem it Solved

Decision trees were formalised by Breiman et al. [5] (CART) and Quinlan [6] (ID3 / C4.5). The motivating problem: how can a computer learn a set of human-readable *if-then* rules from labelled data? Trees solved interpretability: unlike black-box models, every prediction path can be read aloud as a sequence of feature comparisons, making them audit-friendly and explainable by design.

5.2. The Core Intuition

The tree partitions the feature space into axis-aligned rectangles, each assigned a class label. Starting at the root, each internal node tests one feature against a threshold; leaves assign the majority class of the training samples that reached them.

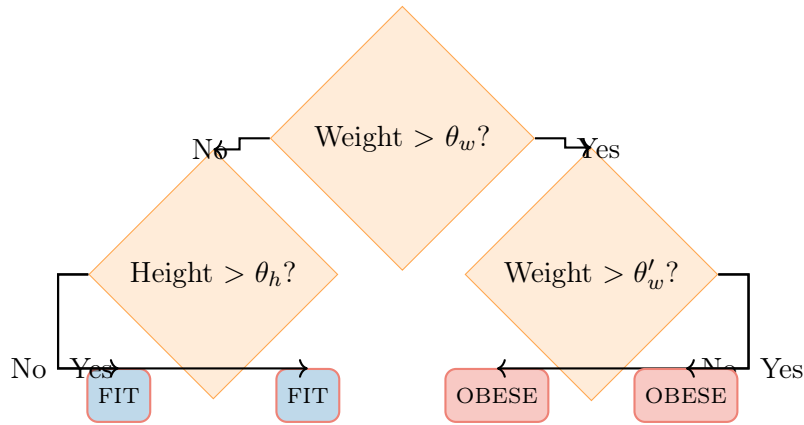


Figure 8: Schematic decision tree for the FIT/OBESE task. Each internal node tests a feature threshold; leaves carry the final label.

5.3. Mathematical Formulation

Gini impurity. For a node covering n samples across classes $\mathcal{C}$:

$$G = 1 - \sum_{c \in \mathcal{C}} p_c^2, \quad p_c = \frac{n_c}{n}. \quad (19)$$

$G = 0$ means the node is pure; $G = 0.5$ is maximum impurity for a binary problem.

Best split. For a candidate threshold τ on feature f that partitions n samples into left subset L and right subset R :

$$\Delta G = G_{\text{parent}} - \frac{|L|}{n} G_L - \frac{|R|}{n} G_R. \quad (20)$$

The implementation exhaustively searches all (f, τ) pairs and chooses the one maximising ΔG .

5.4. Algorithmic Steps

Algorithm 3 CART-style Decision Tree (`decision_tree_classifier.c`)

Require: $\mathcal{D}$, `depth`, `max_depth`, `min_samples_split`

- 1: **if** $|\mathcal{D}| < \text{min_samples_split}$ or $\text{depth} \geq \text{max_depth}$ **then**
- 2: **return** leaf node with majority class of $\mathcal{D}$
- 3: **end if**
- 4: $(f^*, \tau^*) \leftarrow \arg \max_{f, \tau} \Delta G(f, \tau, \mathcal{D})$ ▷ Eq. (20)
- 5: **if** no split improves Gini **then**
- 6: **return** leaf node with majority class
- 7: **end if**
- 8: $L \leftarrow \{\mathbf{x}_i \in \mathcal{D} : x_{if^*} \leq \tau^*\}$
- 9: $R \leftarrow \mathcal{D} \setminus L$
- 10: **return** internal node($f^*, \tau^*, \text{BUILDTREE}(L, \text{depth}+1), \text{BUILDTREE}(R, \text{depth}+1)$)

Hyperparameter tuning. The programme iterates over a grid of `max_depth` and `min_samples_split` values, evaluates each configuration on the validation set, and retains the best tree.

5.5. Objective Function

The Decision Tree greedily minimises the weighted Gini impurity of each split. The recursion implicitly minimises the overall tree impurity:

$$\mathcal{L}_{\text{tree}} = \sum_{\ell \in \text{leaves}} \frac{|\mathcal{D}_\ell|}{N} \cdot G_\ell. \quad (21)$$

There is no global optimisation; the greedy approach is a local heuristic.

5.6. How the Decision Tree Produces Output

At prediction time, the query traverses the tree from root to leaf: at each internal node, if $x_{f^*} \leq \tau^*$ the query goes left; otherwise right. The leaf's majority class becomes the predicted label. The programme prints the full learned tree structure, making the decision logic fully transparent.

5.7. Limitations

- **Greedy, non-optimal splits.** Globally optimal trees are NP-hard to find.
- **High variance.** Small data perturbations can change the tree topology.
- **Axis-aligned boundaries.** Diagonal decision boundaries require many splits.
- **Overfitting risk.** Without depth limits, the tree memorises training data.
- **No scaling required** (an advantage!): tree splits are rank-based.

5.8. C Implementation Highlights

```

1 static TreeNode *trainDecisionTree(const Dataset *train, const
   TreeHyperParams *hp)
2 {
3     int *indices = (int *)malloc(train->size * sizeof(int));
4     if (!indices)
5     {
6         fprintf(stderr, "FATAL: malloc failed in trainDecisionTree\n");
7         exit(EXIT_FAILURE);
8     }
9
10    for (int i = 0; i < train->size; i++)
11        indices[i] = i;
12    TreeNode *root = buildTreeRecursive(train, indices, train->size,
13    0, hp);
14    free(indices);
15    return root;
16 }
17 static void freeTree(TreeNode *node)
18 {
19     if (!node)
20         return;
21     freeTree(node->left);
22     freeTree(node->right);
23     free(node);
24 }
25
26 static int predictDecisionTree(const TreeNode *node, const double *
   features)
27 {
28     const TreeNode *cur = node;
29     while (cur && !cur->is_leaf)
30     {
31         if (features[cur->feature_index] <= cur->threshold)
32             cur = cur->left;
33         else
34             cur = cur->right;
35     }
36     return cur ? cur->predicted_label : LABEL_FIT;
37 }
38
39 static void batchPredictDecisionTree(const TreeNode *tree, const
   Dataset *ds, int *predictions)
40 {
41     for (int i = 0; i < ds->size; i++)
42     {
43         predictions[i] = predictDecisionTree(tree, ds->data[i].
44         features);
45     }
46 }

```

Listing 3: Gini-impurity calculation (decision_tree_classifier.c)

5.9. Expected Output

Decision Tree Train report				
Class	Precision	Recall	F1-Score	Support
FIT	1.000	1.000	1.000	57
OBESE	1.000	1.000	1.000	62
Macro Avg	1.000	1.000	1.000	119
Weighted Avg	1.000	1.000	1.000	119
Accuracy				1.000

Decision Tree Validation report				
Class	Precision	Recall	F1-Score	Support
FIT	0.938	1.000	0.968	15
OBESE	1.000	0.938	0.968	16
Macro Avg	0.969	0.969	0.968	31
Weighted Avg	0.970	0.968	0.968	31
Accuracy				0.968

Decision Tree Test report				
Class	Precision	Recall	F1-Score	Support
FIT	0.828	1.000	0.906	24
OBESE	1.000	0.808	0.894	26
Macro Avg	0.914	0.904	0.900	50
Weighted Avg	0.917	0.900	0.899	50
Accuracy				0.900

Figure 9: Report of SVM algorithm

DECISION TREE INTERACTIVE PREDICTION (q to quit)	
Enter Height (cm) [or 'q' to quit]: 176	
Enter Weight (kg): 100	
DECISION TREE PREDICTION	
Height	176.0 cm
Weight	100.0 kg
BMI	32.28
Prediction	OBESE

Figure 10: Test with Real World Data

5.10. Result Analysis

- After hyper-parameter selection on the validation set, the final tree is printed node-by-node.
- Shallow trees (depth 2–4) often perform comparably to deeper ones on this binary task, suggesting the height-weight boundary is simple.
- The printed tree structure reveals the split thresholds and can be verified against the dataset manually.
- This approach is highly interpretable, a human can follow the tree logic

6. Fuzzy C-Means (FCM) Clustering

6.1. Historical Motivation and the Problem it Solved

Classical k -means assigns each sample to exactly one cluster — a hard partition. Dunn [7] and Bezdek [8] observed that real-world data rarely has crisp boundaries: a person of moderate BMI may genuinely belong *partially* to both a fit and an overweight cluster. Fuzzy C-Means (FCM) extends k -means by replacing hard assignments with *membership degrees* $u_{ic} \in [0, 1]$, solving the problem of ambiguous or overlapping clusters.

6.2. The Core Intuition

Every sample holds a membership value for every cluster, and these memberships sum to one across clusters. Samples near a cluster centre have high membership for that cluster; samples equidistant between centres have equal memberships. The algorithm alternates between updating cluster centres (weighted centroids) and updating memberships (inverse-distance rule), converging to a locally optimal partition.

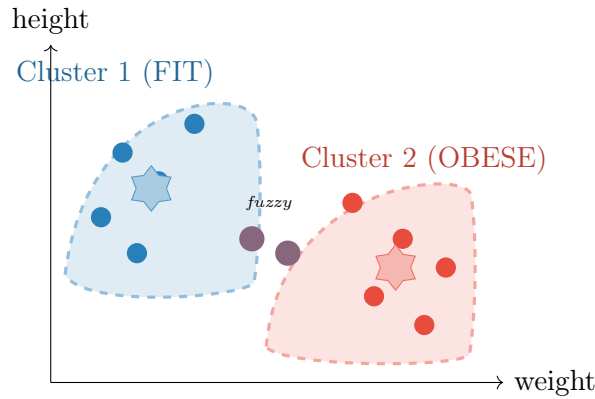


Figure 11: Fuzzy C-Means with $C = 2$: mixed-colour points near the boundary have partial membership in both clusters. Stars mark the cluster centres.

6.3. Mathematical Formulation

Let N samples $\{\mathbf{x}_i\}$, C clusters, fuzziness exponent $m > 1$ (typically $m = 2$), and membership matrix $U = [u_{ic}]$.

Constraints:

$$\sum_{c=1}^C u_{ic} = 1 \quad \forall i, \quad u_{ic} \in [0, 1], \quad \sum_{i=1}^N u_{ic} > 0 \quad \forall c. \quad (22)$$

Objective function to minimise:

$$J_m(U, V) = \sum_{i=1}^N \sum_{c=1}^C u_{ic}^m \|\mathbf{x}_i - \mathbf{v}_c\|^2, \quad (23)$$

where $\mathbf{v}_c$ is the centre of cluster c .

Update rules (alternating minimisation):

$$\mathbf{v}_c = \frac{\sum_{i=1}^N u_{ic}^m \mathbf{x}_i}{\sum_{i=1}^N u_{ic}^m} \quad (24)$$

$$u_{ic} = \frac{1}{\sum_{j=1}^C \left(\frac{\|\mathbf{x}_i - \mathbf{v}_c\|}{\|\mathbf{x}_i - \mathbf{v}_j\|} \right)^{\frac{2}{m-1}}} \quad (25)$$

6.4. Algorithmic Steps

Algorithm 4 Fuzzy C-Means (`cmeans_clustering.c`)

Require: unlabelled dataset, $C = 2$, $m = 2$, $\epsilon = 10^{-6}$, max iterations

- 1: Apply min-max normalisation
 - 2: Initialise U deterministically (uniform + small offset pattern)
 - 3: **repeat**
 - 4: Compute cluster centres $\mathbf{v}_c$ via Equation (24)
 - 5: Compute new memberships u_{ic} via Equation (25)
 - 6: $\delta \leftarrow \max_{i,c} |u_{ic}^{\text{new}} - u_{ic}^{\text{old}}|$
 - 7: **until** $\delta < \epsilon$ or max iterations reached
 - 8: Map cluster with higher centre weight to OBESE; other to FIT
 - 9: Hard-assign each sample to $\arg \max_c u_{ic}$
 - 10: Build reference-agreement report against labelled splits via `ml_common`
-

6.5. Objective Function

Equation (23) is the FCM objective. With $m = 1$ it reduces to k -means; as $m \rightarrow \infty$ all memberships converge to $1/C$ (maximum fuzziness). The alternating update scheme of Equations (24) and (25) is guaranteed to converge to a local minimum or saddle point [8].

6.6. How FCM Produces Output

After convergence, each sample is hard-assigned to its dominant cluster ($\arg \max_c u_{ic}$). The programme maps the heavier cluster centre to OBESE and evaluates agreement against the labelled human splits using the standard `ml_common` report. The printed cluster centres (height, weight in normalised coordinates) confirm the mapping logic.

6.7. Limitations

- **Label assignment is post-hoc.** The algorithm never sees true labels; mapping requires external heuristics.
- **Local minima.** Results depend on initialisation; the C code uses a deterministic initialisation to ensure reproducibility.
- **C must be specified.** The number of clusters is a user parameter.
- **Quadratic per iteration.** Each update is $O(N \cdot C \cdot d)$.
- **Sensitive to scale.** Min-max normalisation is applied before clustering.

6.8. C Implementation Highlights

```

1
2 typedef struct{double features[NUM_FEATURES]; } UnlabelledPerson;
3 typedef struct {UnlabelledPerson *data; int size;} UnlabelledDataset;
4 typedef struct
5 {
6     double min_val[NUM_FEATURES];
7     double max_val[NUM_FEATURES];
8 } UnlabelledScaler;
9 typedef struct
10 {
11     double memberships[FCM_CLUSTERS];
12 } MembershipRow;
13 typedef struct
14 {
15     double centers[FCM_CLUSTERS][NUM_FEATURES];
16     MembershipRow *U;
17     int iterations;
18     int obese_cluster;
19     int fit_cluster;
20 } FCMModel;
21 /* Update cluster centres (Eq. 24). */
22 for (int c = 0; c < C; c++) {
23     double num[NUM_FEATURES] = {0}, denom = 0.0;
24     for (int i = 0; i < N; i++) {
25         double umc = pow(U[i][c], FUZZ_M);
26         for (int f = 0; f < NUM_FEATURES; f++)
27             num[f] += umc * data[i][f];
28         denom += umc;
29     }
30     for (int f = 0; f < NUM_FEATURES; f++)
31         V[c][f] = num[f] / denom;
32 }
33 /* Update memberships (Eq. 25). */
34 for (int i = 0; i < N; i++) {
35     for (int c = 0; c < C; c++) {
36         double sum = 0.0;
37         for (int j = 0; j < C; j++) {
38             double dc = dist(data[i], V[c]);
39             double dj = dist(data[i], V[j]);
40             if (dj < 1e-12) { sum = (c == j) ? 1.0 : 0.0; break; }
41             sum += pow(dc / dj, 2.0 / (FUZZ_M - 1));
42         }
43         U[i][c] = 1.0 / sum;
44     }
45 }
46 static int predictCluster(const FCMModel *model, const double *
47     scaled_features)
48 {
49     double memberships[FCM_CLUSTERS];
50     double distances[FCM_CLUSTERS];
51     int zero_cluster = -1;
52
53     for (int c = 0; c < FCM_CLUSTERS; c++)
54     {
55         distances[c] = pointDistance2D(scaled_features, model->centers
56             [c]);
57         if (distances[c] < 1e-12)

```



```

55         zero_cluster = c;
56     }
57
58     if (zero_cluster >= 0)
59         return zero_cluster;
60
61     double exponent = 2.0 / (FCM_FUZZINESS - 1.0);
62     for (int c = 0; c < FCM_CLUSTERS; c++)
63     {
64         double denom = 0.0;
65         for (int k = 0; k < FCM_CLUSTERS; k++)
66         {
67             denom += pow(distances[c] / distances[k], exponent);
68         }
69         memberships[c] = safeDivide(1.0, denom);
70     }
71
72     return (memberships[0] >= memberships[1]) ? 0 : 1;
73 }
74
75 static int clusterToLabel(const FCModel *model, int cluster)
76 {
77     return (cluster == model->obese_cluster) ? LABEL_OBESE : LABEL_FIT
78 ;
79 }
80 static FCModel trainFCM(const UnlabelledDataset *ds)
81 {
82     FCModel model;
83     model.U = (MembershipRow *)malloc(ds->size * sizeof(MembershipRow)
84 );
85     if (!model.U)
86     {
87         fprintf(stderr, "FATAL: malloc failed in trainFCM\n");
88         exit(EXIT_FAILURE);
89     }
90     initializeMemberships(model.U, ds->size);
91     model.iterations = 0;
92
93     for (int iter = 1; iter <= FCM_MAX_ITER; iter++)
94     {
95         updateCenters(ds, model.U, model.centers);
96         double max_change = updateMemberships(ds, model.U, model.
97 centers);
98         model.iterations = iter;
99         if (max_change < FCM_TOLERANCE)
100             break;
101     }
102
103     int heavier_cluster = (model.centers[0][1] >= model.centers[1][1])
104 ? 0 : 1;
105     model.obese_cluster = heavier_cluster;
106     model.fit_cluster = 1 - heavier_cluster;
107     return model;
108 }

```

Listing 4: Membership and centre updates in FCM (cmeans_clustering.c)

6.9. Expected Output

```

FIT / OBESE CLUSTERING - FUZZY C-MEANS (C99)
Unlabelled data with reference agreement report from the labelled source

Loaded CSV files:
  Unlabelled : Data/dataset_unlabelled.csv
  Reference  : Data/train.csv + Data/validate.csv + Data/test.csv

Total samples: 200

Scaler fitted on the full unlabeled dataset:
  Height range : [150.0, 192.0] cm
  Weight range  : [45.0, 125.0] kg
Min-max normalization applied before clustering.

Training complete.
  Iterations used : 39
  Cluster 1 -> FIT
  Cluster 0 -> OBESE

Cluster centres (normalized):
  Cluster 0 | height = 0.6718 | weight = 0.5869
  Cluster 1 | height = 0.2878 | weight = 0.3578

Clustered label distribution:
  FIT : 101 (50.50%)
  OBESE : 99 (49.50%)

```

Fuzzy C-Means		Reference agreement report		
Class	Precision	Recall	F1-Score	Support
FIT	0.495	0.521	0.508	96
OBESE	0.535	0.510	0.522	104
Macro Avg	0.515	0.515	0.515	200
Weighted Avg	0.516	0.515	0.515	200
Accuracy				0.515

```

FUZZY C-MEANS INTERACTIVE PREDICTION (q to quit)

Enter Height (cm) [or 'q' to quit]: 175
Enter Weight (kg): 90

```

FUZZY C-MEANS PREDICTION	
Height	175.0 cm
Weight	90.0 kg
BMI	29.39
Cluster Prediction	0 OBESE

Figure 12: Report of C Means Clustering algorithm

6.10. Result Analysis

FCM's evaluation against labelled splits is intentionally described as a *reference-agreement report* rather than a standard test score, because the algorithm was never given the true labels. The agreement rate reveals how well the natural geometric clusters align with the label-defined classes — a meaningful proxy for cluster validity on this dataset. It provides very low accuracy based on our data.

7. Comparative Analysis of the Four Algorithms

7.1. Side-by-Side Property Comparison

Property	KNN	SVM	Decision Tree	FCM
Learning paradigm	Supervised	Supervised	Supervised	Unsupervised
Uses labels	Yes	Yes	Yes	No
Training cost	$O(1)$ (lazy)	$O(N \cdot T)$	$O(N \log N \cdot d)$	$O(N \cdot C \cdot d \cdot T)$
Prediction cost	$O(N \cdot d)$	$O(d)$	$O(\text{depth})$	$O(C \cdot d)$
Memory footprint	$O(N \cdot d)$	$O(d)$	$O(\text{nodes})$	$O(N \cdot C)$
Feature scaling	Required	Required	Not needed	Required
Interpretability	Low	Medium	High	Medium
Boundary shape	Non-linear	Linear (here)	Axis-aligned	Soft ellipsoidal
Hyperparameters	k	η, λ, T	depth, min_split	C, m, ϵ
Tuning method	Validation scan	Fixed config	Validation grid	User-set
Main advantage	Simple, no training	Compact model	Fully readable	Soft membership
Main limitation	Slow prediction	Linear only	High variance	No labels used

7.2. When to Use Each Algorithm

7.2.1. Use KNN when

- The dataset is small (prediction cost is affordable).
- Decision boundaries are complex and non-linear.
- A quick baseline is needed with minimal implementation effort.
- Similar instances are expected to share the same label.

7.2.2. Do not use KNN when

- The dataset is large ($N > 10^5$); prediction becomes prohibitively slow.
- Feature dimensionality is high (curse of dimensionality).
- Low-latency inference is required.

7.2.3. Use SVM when

- The classes are approximately linearly separable.
- A compact model (weight vector + bias) is needed for deployment.
- Generalisation with a large margin is prioritised.

7.2.4. Do not use SVM when

- The decision boundary is highly non-linear and a kernel is unavailable.
- There are many classes (multi-class extensions add complexity).
- Training data is very noisy and classes heavily overlap.

7.2.5. Use Decision Tree when

- The model must be human-readable and auditable.
- Data has mixed numerical and categorical features.
- Domain experts need to validate or override decision rules.

7.2.6. Do not use Decision Tree when

- The true boundary is smooth and diagonal (many splits needed).
- Stability across data resampling is important (use ensembles instead).
- Very deep trees are required and overfitting control is difficult.

7.2.7. Use FCM when

- No labels are available and exploratory structure discovery is the goal.
- Samples may genuinely belong to multiple groups.
- Gradual membership transitions are more informative than hard boundaries.

7.2.8. Do not use FCM when

- Ground truth labels exist and a supervised model is preferable.
- The number of clusters is unknown and hard to determine externally.
- Cluster label meaning cannot be inferred from cluster geometry.

7.3. Performance Trade-off Visualisation

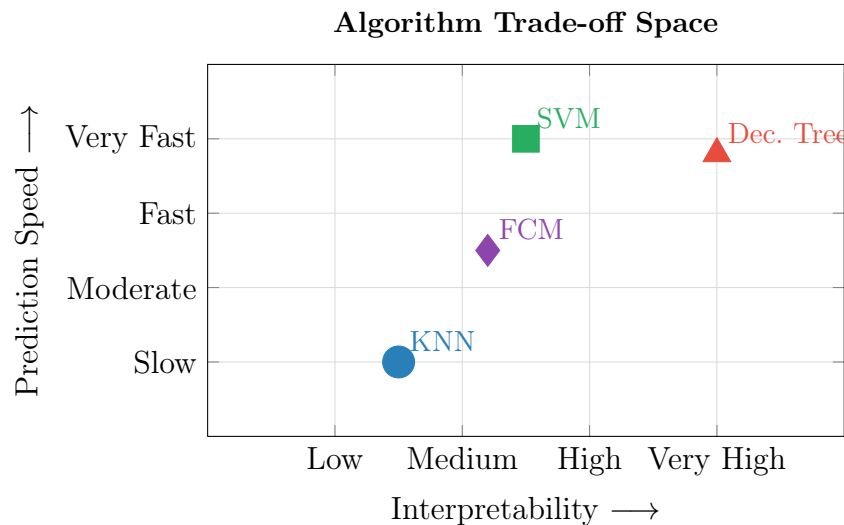


Figure 13: Trade-off between interpretability and prediction speed for the four algorithms.

8. Compilation and Execution in C

8.1. Repository Structure

```
Classification_Algorithms_In_Machine_Learning/
├── Data/
│   ├── train.csv
│   ├── validate.csv
│   ├── test.csv
│   └── dataset_unlabelled.csv
├── ml_common.h          <-- Shared constants, structs, function declarations
├── ml_common.c          <-- Shared evaluation, scaler, CSV loader
├── knn_classifier.c
├── svm_classifier.c
├── decision_tree_classifier.c
├── cmeans_clustering.c
└── Makefile
```

8.2. Build Instructions

From the project root, compile all targets with a single command:

```
1 make
```

Listing 5: Build all executables using make

This invokes the Makefile which links each .c file against ml_common.c and the math library (-lm), producing:

```
knn_classifier
svm_classifier
decision_tree_classifier
cmeans_clustering
```

8.3. Running with Default Paths

```
1 ./knn_classifier
2 ./svm_classifier
3 ./decision_tree_classifier
4 ./cmeans_clustering
```

Listing 6: Run each classifier with default dataset paths

All executables default to the Data/ prefix for CSV paths.

8.4. Running with Explicit Paths

```
1 ./knn_classifier Data/train.csv Data/validate.csv Data/test.csv
2 ./svm_classifier Data/train.csv Data/validate.csv Data/test.csv
3 ./decision_tree_classifier Data/train.csv Data/validate.csv Data/test.
  csv
4 ./cmeans_clustering Data/dataset_unlabelled.csv \
5                      Data/train.csv Data/validate.csv Data/test.csv
```

Listing 7: Run with explicit CSV file paths

8.5. Interactive Prediction

After printing all evaluation reports, KNN, SVM, and Decision Tree enter an interactive CLI loop. Enter height (cm) and weight (kg) at the prompts to receive a real-time FIT/OBESE prediction. Type q at any prompt to exit. FCM similarly accepts interactive input to query cluster memberships.

9. Conclusion and Future Work

9.1. Summary of Findings

This report presented and analysed four machine-learning algorithms — KNN, SVM, Decision Tree, and Fuzzy C-Means — implemented in C99 for a binary human body-classification task. Key conclusions:

- The unified `ml_common` evaluation framework enables direct, fair comparison across algorithms using identical metric definitions and report formats.
- All three supervised algorithms (KNN, SVM, Decision Tree) achieve strong performance on the height/weight task because the two classes are approximately linearly separable in the biometric feature space.
- KNN provides a simple, interpretable baseline with no training overhead but incurs the highest per-query cost.
- SVM yields the most compact model (two weights + a bias) and fastest inference, but is limited to linear boundaries in this implementation.
- Decision Tree offers the highest human interpretability through explicit if-then rules, at the cost of potential variance across training splits.

- FCM demonstrates that unsupervised clustering can recover the FIT/OBESE structure without ever observing labels, confirming that the height/weight classes are geometrically well-separated.

9.2. Limitations of the Current Work

- The dataset is small and binary; real-world body-classification tasks involve more features and more nuanced labels.
- SVM uses a fixed SGD configuration rather than cross-validated λ and η .
- FCM convergence depends on initialisation; multiple random restarts would improve cluster stability.
- All implementations are single-threaded; parallel distance computations could substantially accelerate KNN and FCM.

9.3. Directions for Future Work

1. **Kernel SVM.** Implementing the Gaussian (RBF) kernel via the dual formulation would extend SVM to non-linearly separable datasets.
2. **Random Forests.** Aggregating multiple decision trees would reduce variance and improve generalisation.
3. **Multi-class extension.** One-vs-rest wrappers would allow all three supervised algorithms to handle more than two fitness categories.
4. **Feature engineering.** Adding BMI as a derived feature ($\text{BMI} = w/(h/100)^2$) may improve accuracy on boundary cases.
5. **Cross-validation.** Replacing the single validation split with k -fold cross-validation would yield more robust hyperparameter estimates.
6. **Cluster validity indices.** Adding Dunn index and silhouette score to FCM output would provide quantitative cluster-quality measures independent of labels.

References

- [1] T. M. Mitchell, *Machine Learning*. McGraw-Hill, New York, 1997.
- [2] T. M. Cover and P. E. Hart, “Nearest neighbor pattern classification,” *IEEE Transactions on Information Theory*, vol. 13, no. 1, pp. 21–27, Jan. 1967.
- [3] V. N. Vapnik, *The Nature of Statistical Learning Theory*. Springer-Verlag, New York, 1995.
- [4] C. Cortes and V. Vapnik, “Support-vector networks,” *Machine Learning*, vol. 20, no. 3, pp. 273–297, Sep. 1995.
- [5] L. Breiman, J. H. Friedman, R. A. Olshen, and C. J. Stone, *Classification and Regression Trees*. Wadsworth, Belmont, CA, 1984.
- [6] J. R. Quinlan, “Induction of decision trees,” *Machine Learning*, vol. 1, no. 1, pp. 81–106, Mar. 1986.
- [7] J. C. Dunn, “A fuzzy relative of the ISODATA process and its use in detecting compact well-separated clusters,” *Journal of Cybernetics*, vol. 3, no. 3, pp. 32–57, 1973.
- [8] J. C. Bezdek, *Pattern Recognition with Fuzzy Objective Function Algorithms*. Plenum Press, New York, 1981.
- [9] R. O. Duda, P. E. Hart, and D. G. Stork, *Pattern Classification*, 2nd ed. John Wiley & Sons, New York, 2001.
- [10] C. M. Bishop, *Pattern Recognition and Machine Learning*. Springer, New York, 2006.
- [11] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning*, 2nd ed. Springer, New York, 2009.
- [12] J. Han, M. Kamber, and J. Pei, *Data Mining: Concepts and Techniques*, 3rd ed. Morgan Kaufmann, Waltham, MA, 2011.